

STACK :-

- * A Stack is a Linear data structure
- * The Java Collection framework provides a Stack class, which implements a Stack data structure.
- * The class is based on the basic principle of LIFO (Last in first out)
- * Stack has one end, whereas queue has two ends (front & rear)
- * A Stack is a container, in which insertion and deletion can be done from the end (one) known as the top of the stack.

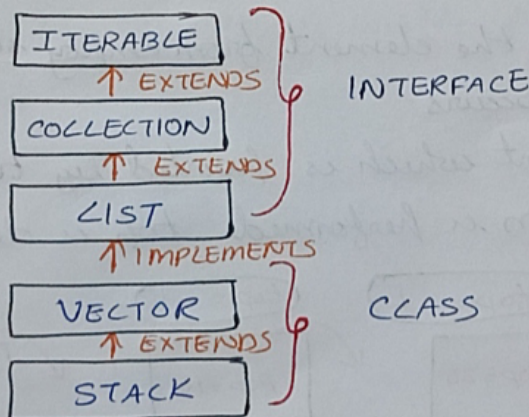
NOTE :-

- * A Stack is an Abstract Data type with a pre-defined capacity, which means that it can store elements of limited size.

Operations on the stack :-

- ① push() :- When we insert an element in a stack then the operation is known as push. If stack is full overflow condition occurs.
- ② pop() :- When we delete an element from stack, the operation is called as pop(). If stack is empty means no element exists in the stack, this state is known as an underflow state.
- ③ peek() :- It returns the element at a given position.
- ④ count() :- It returns the total number of elements available in a stack.
- ⑤ change() :- It changes the element at the given position.
- ⑥ display() :- It prints all the elements available in the stack.

Hierarchy :-



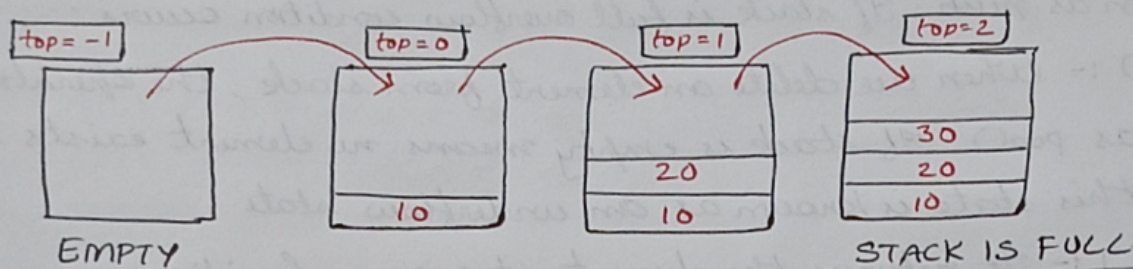
How to create a stack?

In order to create a stack, we must import `java.util.Stack` package and use the `Stack()` constructor of this class. The below example creates an empty stack.

```
Stack<E> stack = new Stack<E>();
```

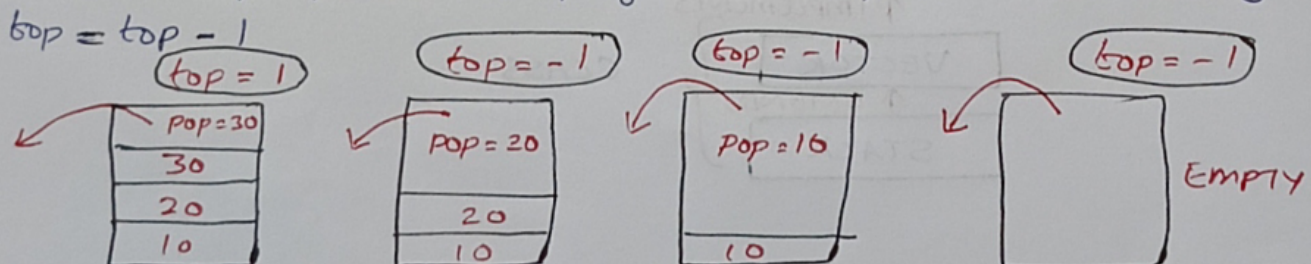
PUSH OPERATION :-

- * Before inserting an element in the a stack, we check whether the stack is full.
- * If we try to insert element in a stack, and the stack is full, then overflow condition occurs.
- * When we initialized a stack, we set the value of `top` as `-1` to check that stack is empty.
- * The elements will be inserted until we reach the max size of the stack. $top = top + 1$



POP OPERATION :-

- * Before deleting the element from the stack, we check whether the stack is empty.
- * If we try to delete the element from empty stack, then underflow condition occurs.
- * First access the element which is pointed by `top`.
- * Once the `top` operation is performed, `top` is decremented by 1.



Applications of Stack :-

Recursion :- The recursion means that the function is calling itself again to maintain the previous states, the compiler creates a system stack in which all previous records of function are maintained.

DFS (Depth first search) :- This search is implemented on a graph, graph uses stack d.s

Back tracking :- If we have to create a path to solve maze problem, if we are moving in particular path and we realise that we came on the wrong way. In order to come at beginning of the path to create a new path, we use stack d.s

Memory management :- The stack manages the memory. The memory is assigned in the continuous memory blocks.

NOTATIONS :-

INFIX :- The operator is placed between its operands. This is the most common mathematical notation used by humans. For example, the expression " $2+3$ " is an infix expression, where the operator "+" is placed between the operands "2" and "3".

PREFIX :- It is also known as Polish notation, the operator precedes its operands. This differs from the more common infix notation, where the operator is placed between its operands.

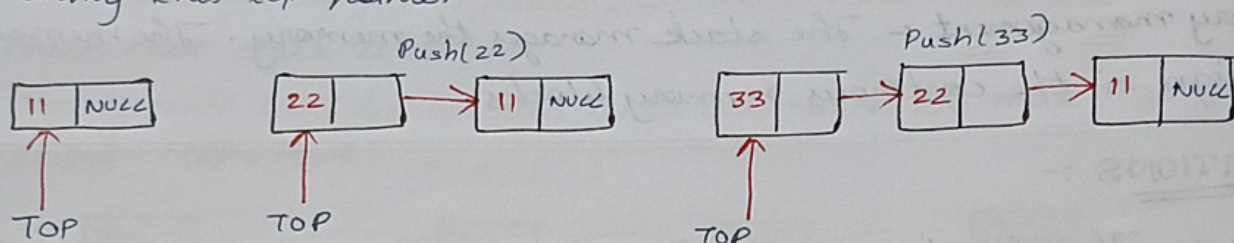
* In prefix notation, the operator is written first, followed by its operands. Example " $a+b$ " would be written as "+ab".

POSTFIX :- It is also known as Reverse Polish notation, where the operator follows its operands. This differs from the more common infix notation, where the operator is placed between its operands.

* In postfix notation, operands are written first, " $5+2$ " would be written as " $52+$ " in postfix notation.

Implement a stack using singly linked list :-

- * To implement a stack using a singly linked list, we need to ensure that all operations follow the LIFO (Last in, First Out) principle.
- * This means that the most recently added element is always the first one to be removed.
- * In this approach, we use a singly linked list, where each node contains data and a reference to the next node.
- * To manage the stack, we maintain a top pointer that always points to the most recent (topmost) node in the stack.
- * The key stack operations - push, pop, and peek can be performed using this top pointer.



- * In the stack implementation, a stack contains a top pointer, which is the "head" of the stack where pushing & popping items happens at the head of the list.
- * The main advantage of using a linked list over arrays is that it is possible to implement a stack that can shrink or grow as much as needed.
- * Using an array will put a restriction on the maximum capacity of the array which can lead to stack overflow.

Q/A

- 01) Valid Parentheses * * * * *
- 02) Min Stack * * * * *
- 03) Stack *
- 04) Largest Rectangular Area in Histogram * *
- 05) Next Greater Element - I * * * *
- 06) Next Greater Element - II *
- 07) Maximal Size Binary Matrix Containing -1 *
- 08) Infix Evaluation *
- 09) Previous Smaller Element * *
- 10) Celebrity Problem * *
- 11) Get Min Element from Stack * *
- 12) Stock Span * * *
- 13) Max Area in Histogram * *
- 14) K-stacks in a single array (Stack & Queue) *
- 15) Daily Temperatures * *